Министерство образования Республики Беларусь

Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Модели данных и системы управления базами данных»

**ОТЧЁТ**
к лабораторной работе №4
на тему
**«REDIS»**
БГУИР 6-05-0612-02 23

Выполнил студент группы 353503
СЕБЕЛЕВ Дмитрий Юрьевич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ИОВИЧ Евгения Александровна

_____
(дата, подпись преподавателя)

Минск 2026

## 1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

1 Реализовать миграцию данных из формата *CSV* в хранилище *Redis*. Необходимо спроектировать структуру ключей и выбрать подходящие типы данных для эффективного доступа.

2 Реализовать бизнес-логику для получения последних заказов пользователя с отображением связанных позиций заказа и данных о товарах.

3 Реализовать получение топа товаров по объему продаж и по выручке. Внедрить механизм кэширования результатов с заданным временем жизни (*TTL*).

4 Реализовать фильтрацию товаров по категориям и диапазону цен с использованием упорядоченных множеств (*Sorted Sets*).

5 Разработать систему рекомендаций на основе поиска похожих пользователей, используя теоретико-множественные операции (*SINTER*, *SUNION*, *SDIFF*).

6 Реализовать аналогичные запросы в реляционной СУБД (*PostgreSQL*) для демонстрации различий между подходами к обработке данных.

# 2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

## 2.1 Хранилище Redis

*Redis* (*Remote Dictionary Server*) представляет собой высокопроизводительную нереляционную систему управления базами данных класса *Key-Value*, работающую в оперативной памяти. Основной особенностью системы является поддержка сложных структур данных и атомарность операций над ними.

## 2.2 Типы данных

В ходе работы использовались следующие типы данных:

– *Hash* — ассоциативный массив, применяемый для хранения объектов (пользователи, товары, заказы). Позволяет обращаться к отдельным полям объекта без необходимости чтения всей структуры;

– *Set* — неупорядоченная коллекция уникальных строк. Эффективно используется для хранения связей (например, позиции в заказе) и выполнения операций пересечения или объединения;

– *Sorted Set* (*ZSet*) — коллекция уникальных элементов, где каждый элемент ассоциирован с числовым весом (*score*). Позволяет выполнять выборку по диапазону весов и получать элементы в отсортированном виде;

– *List* — упорядоченный список строк. Используется для хранения хронологических данных, таких как история заказов пользователя.

## 2.3 Стратегии кэширования

Кэширование применяется для снижения нагрузки на систему при выполнении ресурсоемких агрегационных запросов. В работе реализована стратегия *Cache Aside*, где данные сначала запрашиваются из кэша, а при их отсутствии — вычисляются и сохраняются в кэш с установкой времени актуальности (*TTL*).

# 3 ВЫПОЛНЕНИЕ РАБОТЫ

## 3.1 Соглашение об именовании ключей

Для обеспечения структурированности данных в плоском пространстве ключей использован префиксный подход с разделением двоеточиями:

- *user:{id}* — основные данные пользователя;
- *user:{id}:orders* — список идентификаторов заказов пользователя;
- *product:{id}* — параметры товара;
- *order:{id}* — заголовок заказа;
- *order:{id}:items* — множество позиций в заказе.

## 3.2 Процесс миграции данных

Миграция осуществлялась путем последовательного чтения файлов *CSV* и записи данных в соответствующие структуры *Redis*. Для ускорения доступа к агрегатам при загрузке данных формировались дополнительные индексы в виде упорядоченных множеств (например, *products:by_price*).

Листинг 3.1 – Миграция данных о товарах и создание индексов по цене

```
// Создание Hash для товара
rdb.HSet(ctx, fmt.Sprintf("product:%s", productID), map[string]any{
    "product_id":  productID,
    "name":        row[1],
    "category_id": categoryID,
    "price":       price,
})

// Добавление в индекс по цене (Sorted Set)
rdb.ZAdd(ctx, "products:by_price", redis.Z{
    Score:  priceFloat,
    Member: productID,
})
```

## 3.3 Получение последних заказов

Метод *GetRecentOrders* выполняет сопоставление связанных сущностей. Сначала извлекаются идентификаторы заказов из списка пользователя (*List*), затем для каждого заказа запрашиваются его атрибуты из *Hash*, позиции из множества (*Set*) и детальная информация о товарах.

Листинг 3.2 – GetRecentOrders в Redis

```
orderIDs, _ := s.rdb.LRange(ctx, key, 0, limit-1).Result()
for _, id := range orderIDs {
    s.rdb.HGetAll(ctx, fmt.Sprintf("order:%s", id)).Scan(&order)
    orderItemIDs, _ := s.rdb.SMembers(ctx, fmt.Sprintf("order:
```

```
%s:items", id)).Result()
    // ...
}
```

Аналогичный запрос в PostgreSQL:

Листинг 3.3 – GetRecentOrders в PostgreSQL

```
SELECT order_id, user_id, created_at, status
FROM orders
WHERE user_id = $1
ORDER BY created_at DESC
LIMIT $2
```

### 3.4 Кэширование популярных товаров

Метод *GetTopProducts* возвращает список наиболее продаваемых товаров. Для оптимизации используется строковый ключ с параметрами запроса. Если данные присутствуют в кэше, они возвращаются немедленно. В противном случае выполняется выборка из *ZSet* (*ZREVRANGE*), данные сериализуются в *JSON* и сохраняются в кэш на 60 секунд.

Листинг 3.4 – Реализация кэширования с TTL

```
cached, err := s.rdb.Get(ctx, cacheKey).Result()
if err == nil {
    s.CacheHits++
    json.Unmarshal([]byte(cached), &stats)
    return stats, nil
}

// Выборка из индекса при промахе кэша
results, _ := s.rdb.ZRevRangeWithScores(ctx, zsetKey, 0, n-1).Result()
// ... формирование результата ...

data, _ := json.Marshal(stats)
s.rdb.Set(ctx, cacheKey, data, 60*time.Second)
```

### 3.5 Фильтрация товаров по категориям

Для реализации функции *GetProductsByCategory* использовались упорядоченные множества, где ключом выступает идентификатор категории, а весом — цена товара. Это позволяет эффективно ограничивать выборку диапазоном цен с помощью команды *ZRANGEBYSCORE*.

Листинг 3.5 – GetProductsByCategory в Redis

```
productIDs, _ := s.rdb.ZRangeArgs(ctx, redis.ZRangeArgs{
    Key:     fmt.Sprintf("category:%d:products", categoryID),
```

```
    ByScore: true,
    Start:   minPrice,
    Stop:    maxPrice,
}).Result()
```

Аналогичный запрос в PostgreSQL:

Листинг 3.6 – GetProductsByCategory в PostgreSQL

```
SELECT product_id, name, category_id, price
FROM products
WHERE category_id = $1
  AND price >= $2
  AND price <= $3
ORDER BY price ASC
```

### 3.6 Система рекомендаций похожим пользователям

Алгоритм рекомендаций основан на анализе покупок. Процесс включает следующие этапы:

1 Поиск множества товаров, купленных текущим пользователем.

2 Поиск других пользователей, имеющих пересечение по купленным товарам (*SINTER*).

3 Объединение всех товаров, купленных этими пользователями (*SUNION*).

4 Исключение товаров, которые текущий пользователь уже приобрел (*SDIFF*).

Листинг 3.7 – Выполнение теоретико-множественных операций для рекомендаций

```
// Поиск пересечения (похожие пользователи)
commonProducts, _ := s.rdb.SInter(ctx, userPurchasedKey,
otherUserKey).Result()

// Объединение товаров от похожих пользователей
allSimilarProducts, _ := s.rdb.SUnion(ctx,
similarUserKeys...).Result()

// Вычитание уже купленных товаров
recommendations, _ := s.rdb.SDiff(ctx, tempKey,
userPurchasedKey).Result()
```

Аналогичный запрос в PostgreSQL с использованием CTE:

Листинг 3.8 – Рекомендации в PostgreSQL

```
WITH target_user_products AS (
    SELECT DISTINCT product_id
```

```
    FROM order_items oi
    JOIN orders o ON oi.order_id = o.order_id
    WHERE o.user_id = $1
),
similar_users AS (
    SELECT DISTINCT o.user_id
    FROM orders o
    JOIN order_items oi ON o.order_id = oi.order_id
    WHERE oi.product_id IN (SELECT product_id FROM
target_user_products)
      AND o.user_id != $1
),
recommended_products AS (
    SELECT DISTINCT oi.product_id
    FROM order_items oi
    JOIN orders o ON oi.order_id = o.order_id
    WHERE o.user_id IN (SELECT user_id FROM similar_users)
      AND oi.product_id NOT IN (SELECT product_id FROM
target_user_products)
)
SELECT p.product_id, p.name, p.category_id, p.price
FROM products p
JOIN recommended_products rp ON p.product_id = rp.product_id
LIMIT $2
```

## ВЫВОД

В ходе выполнения лабораторной работы были изучены принципы работы с высокопроизводительным хранилищем данных *Redis*. Были приобретены следующие навыки:

– моделирование данных с использованием различных структур *Redis* (*Hash*, *Set*, *Sorted Set*, *List*) для эффективного решения бизнес-задач;

– проектирование систем индексации в нереляционных базах данных для поддержки сложных выборок и фильтрации;

– реализация механизмов кэширования с управлением временем жизни данных для оптимизации производительности приложения;

– применение теоретико-множественных операций для построения аналитических систем и механизмов рекомендаций.

Разработанное решение демонстрирует преимущества использования *In-Memory* хранилищ для задач, требующих минимального времени отклика и высокой масштабируемости при обработке типичных паттернов доступа к данным.